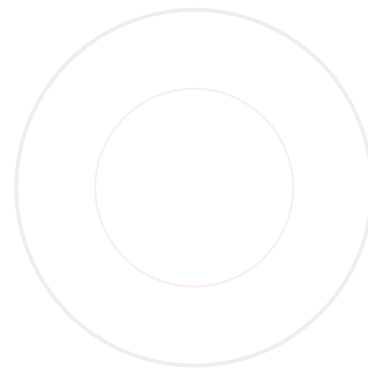


DEPARTEMENT 1

# Architecture

## Systeme

Administration et Gestion des Carrieres  
Sections 0 et 1 : Introduction et Architecture Commune



### CONTENU DE CE DOCUMENT

- Section 0 - Introduction
- Section 1 - Architecture Commune
  - Section 2 - Service 1.1 (a venir)
  - Section 3 - Service 1.2 (a venir)
  - Section 4 - Service 1.3 (a venir)
  - Sections 5-7 (a venir)

Projet : SIRH Multi-Tenant Stack : Supabase + Cloudflare  
Domaines : D2, D4, D5, D12, D21, D23, D24 Version : 1.0

# Table des matieres

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>Section 0 - Introduction</b>                          | <b>2</b> |
| 0.1 Objectif du document . . . . .                       | 2        |
| 0.2 Perimetre fonctionnel . . . . .                      | 2        |
| 0.3 Principes directeurs . . . . .                       | 3        |
| 0.4 Acteurs et parties prenantes . . . . .               | 4        |
| 0.5 Vue fonctionnelle globale . . . . .                  | 5        |
| 0.6 Glossaire . . . . .                                  | 5        |
| <b>Section 1 - Architecture Commune</b>                  | <b>6</b> |
| 1.1 Vue d'ensemble de l'architecture . . . . .           | 6        |
| 1.2 Modele de donnees partage . . . . .                  | 7        |
| 1.2.1 Conventions de nommage . . . . .                   | 8        |
| 1.3 Authentification et Autorisation . . . . .           | 9        |
| 1.3.1 Roles et permissions . . . . .                     | 10       |
| 1.4 Multi-tenancy par Row Level Security . . . . .       | 11       |
| 1.4.1 Activation et gestion des politiques RLS . . . . . | 12       |
| 1.5 Architecture evenementielle inter-domaines . . . . . | 12       |
| 1.6 Stockage et Cache . . . . .                          | 14       |
| 1.6.1 Strategie d'invalidation du cache . . . . .        | 15       |

# Section 0 - Introduction

## 0.1 Objectif du document

Le present document constitue le premier volet de l'architecture systeme du Departement 1 (Administration et Gestion des Carrieres) de la Direction des Ressources Humaines. Il a pour vocation de definir les fondements techniques, organisationnels et fonctionnels sur lesquels reposera l'ensemble des sept domaines composant ce departement. Ce document s'adresse aux architectes logiciels, aux developpeurs, aux chefs de projet et aux decideurs RH qui souhaitent comprendre la vision globale du systeme avant d'entrer dans les details d'implementation de chaque domaine specifique.

L'architecture presentee ici repose sur un choix technologique delibere : l'utilisation combinee de Supabase comme backend de donnees et de Cloudflare comme couche edge de distribution et de traitement. Ce duo offre un modele multi-tenant performant, securise et evolutif, capable de repondre aux besoins varies des PME comme des grandes entreprises. Le systeme est concu pour gerer approximativement 620 tables reparties sur 31 domaines fonctionnels, tout en maintenant une coherence technique et une isolation des donnees rigoureuse entre les differents tenants (organisations clientes).

Ce document couvre deux sections principales. La Section 0 (Introduction) pose le cadre general : objectifs, perimetre fonctionnel, principes directeurs, acteurs impliquees et glossaire des termes cles. La Section 1 (Architecture Commune) detaille les composants techniques partagees par l'ensemble des domaines du departement : modele de donnees, systeme d'authentification et d'autorisation, strategie de multi-tenancy via les Row Level Security (RLS), architecture evenementielle inter-domaines, et systemes de stockage et de cache. Les sections subsequentes (2 a 7), qui feront l'objet de documents separes, traiteront respectivement de chaque service et des interconnexions entre domaines.

## 0.2 Perimetre fonctionnel

Le Departement 1 englobe la totalite du cycle de vie administrative et professionnelle des collaborateurs au sein de l'organisation. Il est structure en trois services, eux-memes decomposes en sept domaines fonctionnels. Chaque domaine couvre un champ d'activite precis et dispose de son propre jeu de tables dans la base de donnees, tout en partageant un socle commun de donnees referencees avec les autres departements de la DRH.

| Service                               | Domaine                     | Cod<br>e | Description                                                                |
|---------------------------------------|-----------------------------|----------|----------------------------------------------------------------------------|
| Service 1.1<br>Gestion Administrative | Gestion Admin. du Personnel | D2       | Donnees de base des employes, etat civil, situation familiale, coordonnees |
|                                       | Gestion des Salaires        | D5       | Elements de remuneration, primes, indemnites, bulletins de paie            |

| Service                                        | Domaine                                    | Cod e | Description                                                                  |
|------------------------------------------------|--------------------------------------------|-------|------------------------------------------------------------------------------|
|                                                | Administration du Personnel                | D12   | Entrees/sorties, conges, absences, autorisations, disciplines                |
| <b>Service 1.2</b><br>Organisation du Travail  | Gestion du Temps                           | D4    | Pointages, planning, horaires, comptes d'heures, equilibre vie pro/perso     |
|                                                | Organigramme                               | D23   | Structure hierarchique, nomenclatures, entites organisationnelles            |
|                                                | Effectifs                                  | D24   | Previsions, mouvements, tableaux de bord sociaux, demographic RH             |
| <b>Service 1.3</b><br>Cartographie des Metiers | Classification et Cartographie des Emplois | D21   | Fiches de poste, reperes de classification, competences, grilles de carriere |

Tableau 0.1 - Mapping Services / Domaines du Departement 1

### 0.3 Principes directeurs

L'architecture du Departement 1 repose sur un ensemble de principes directeurs qui guident l'ensemble des decisions de conception et d'implementation. Ces principes ont ete definis en amont du projet et s'appliquent de maniere uniforme a l'ensemble des 31 domaines de la DRH, garantissant ainsi la coherence globale du systeme quel que soit le departement ou le service concerne.

| Principe                    | Description                                                                                                                                                                                | Application                                                                               |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Multi-tenant par RLS</b> | Un seul projet Supabase pour tous les clients. L'isolation des donnees est assuree par des politiques de Row Level Security (RLS) basees sur le champ tenant_id present dans chaque table. | Chaque requete SQL est automatiquement filtree par le tenant de l'utilisateur connecte.   |
| <b>Domain-Driven Design</b> | Chaque domaine fonctionnel correspond a un package dans le monorepo. Les migrations SQL sont versionnees par domaine, et les frontieres entre domaines sont explicites.                    | packages/domain-d02-gestion-admin/ contient ses propres migrations, types, et composants. |
| <b>API-First</b>            | Toute donnee transite par des API typees (Supabase RPC + Cloudflare Workers). Aucun composant frontend n'accede directement a la base de donnees.                                          | Les Workers Cloudflare exposent les endpoints /api/v1/... et valident les permissions.    |

| Principe                   | Description                                                                                                                                                | Application                                                                                   |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Evenementiel</b>        | Les domaines communiquent via des evenements (PostgreSQL Triggers + Supabase Realtime + Cloudflare Workers Events). Pas de couplage direct entre domaines. | Une modification de salaire (D5) emit un evenement qui met a jour l'historique carriere (D2). |
| <b>Securite en couches</b> | Authentification JWT via Supabase Auth, autorisation RLS au niveau base, validation des inputs dans les Workers, et protection CORS/CSRF au niveau edge.   | 4 couches de securite independentes pour une defense en profondeur.                           |
| <b>Performance Edge</b>    | Les donnees frequentes (dictionnaires, sessions) sont mises en cache dans Cloudflare KV/D1. Les assets statiques sont servis par le CDN Cloudflare.        | Latence de premier chargement inferieure a 2 secondes meme sur reseaux mobiles.               |

Tableau 0.2 - Principes directeurs de l'architecture

## 0.4 Acteurs et parties prenantes

Le systeme d'information RH est utilise par plusieurs categories d'acteurs, chacun ayant des besoins specifiques en termes de fonctionnalites, de permissions et d'interfaces. La comprehension de ces acteurs est essentielle pour dimensionner correctement les mecanismes d'authentification, d'autorisation et d'interface utilisateur de l'architecture.

| Acteur                        | Profil                                              | Besoins principaux                                                | Niveau d'accès                             |
|-------------------------------|-----------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------|
| <b>DRH / Cadre dirigeant</b>  | Decisionnel, vue transversale sur tous les domaines | Tableaux de bord KPI, rapports statistiques, pilotage strategique | Lecture globale + Ecriture restreinte      |
| <b>Responsable de service</b> | Manager operationnel d'un service ou departement    | Gestion de son equipe, validation des conges, plannings           | Lecture/Ecriture sur son perimetre         |
| <b>Agent RH</b>               | Executeur operationnel des processus RH quotidiens  | Saisie des dossiers, gestion des entrees/sorties, bulletins       | Lecture/Ecriture sur tous les domaines DRH |
| <b>Collaborateur</b>          | Salarie de l'organisation, utilisateur final        | Consultation de son dossier, demande de conge, fiche de paie      | Lecture/Ecriture restreinte (son dossier)  |
| <b>Administrateur systeme</b> | IT, gestion de la plateforme technique              | Configuration des tenants, gestion des roles, monitoring          | Acces technique complet                    |

Tableau 0.3 - Acteurs et parties prenantes du systeme

## 0.5 Vue fonctionnelle globale

La vue fonctionnelle globale presente les grandes fonctions du Departement 1 et leurs interdependances. Le systeme est organise selon le principe de separation des responsabilites : chaque service traite un axe metier specifique, mais tous partagent un socle de donnees communes (employes, structures, tenants) qui assure la coherence transversale de l'information RH. Ainsi, lorsqu'un collaborateur est recrute (D2), son poste est automatiquement reference dans la structure organisationnelle (D23), son salaire est initialise dans le systeme de paie (D5), et sa fiche de poste peut etre associee a une classification (D21).

Cette vue met en evidence le caractere fortement integre des domaines du Departement 1. Contrairement a d'autres departements de la DRH qui peuvent fonctionner de maniere plus autonome, le Departement 1 constitue le coeur operationnel du SIRH. Les donnees qu'il produit et qu'il gere alimentent l'ensemble des autres departements (Paie, Formation, Securite, etc.). C'est pourquoi l'architecture commune presentee en Section 1 revet une importance particuliere : elle definit les mecanismes de partage, de securisation et de synchronisation des donnees qui garantissent la fiabilite de l'ensemble du systeme d'information RH.

## 0.6 Glossaire

Ce glossaire rassemble les termes techniques et metiers utilises dans l'ensemble du document d'architecture du Departement 1. Il constitue une reference commune pour tous les intervenants du projet, qu'ils soient techniques, fonctionnels ou decisionnels.

| Terme                              | Definition                                                                                                                                                            |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tenant</b>                      | Organisation cliente hebergee sur la plateforme. Chaque entreprise utilisatrice du SIRH est un tenant distinct, avec ses propres donnees isolees via le systeme RLS.  |
| <b>RLS (Row Level Security)</b>    | Mecanisme natif de PostgreSQL qui permet de definir des regles d'accès au niveau de chaque ligne d'une table. Utilise ici pour isoler les donnees de chaque tenant.   |
| <b>Domaine</b>                     | Unite fonctionnelle du SIRH correspondant a un sous-ensemble coherent de processus RH. Le Departement 1 comprend 7 domaines (D2, D4, D5, D12, D21, D23, D24).         |
| <b>Edge Function</b>               | Fonction executee sur le reseau de distribution de contenu (CDN) de Cloudflare, au plus pres de l'utilisateur final, pour reduire la latence.                         |
| <b>Worker</b>                      | Service serverless execute sur l'infrastructure edge de Cloudflare. Les Workers servent d'API gateway, de transformateur de donnees et de coordonnateur d'evenements. |
| <b>Monorepo</b>                    | Referentiel de code unique contenant l'ensemble des packages de l'application, organises par domaine fonctionnel et gere par Turborepo.                               |
| <b>PWA (Progressive Web App)</b>   | Application web progressive qui fonctionne comme une application native, avec support hors-ligne, notifications push et installation sur l'ecran d'accueil.           |
| <b>RPC (Remote Procedure Call)</b> | Mecanisme de Supabase permettant d'executer des fonctions SQL stockees comme des endpoints API, avec la meme securite RLS que les requetes directes.                  |

| Terme                       | Definition                                                                                                                                                 |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JWT (JSON Web Token)</b> | Standard de jeton d'authentification compact et securise. Supabase Auth emet des JWT contenant l'identite utilisateur, le tenant_id et les roles.          |
| <b>Migration SQL</b>        | Fichier de modification incrementale du schema de base de donnees. Les migrations sont versionnees, ordonnees chronologiquement et organisees par domaine. |

*Tableau 0.4 - Glossaire des termes cles*

## Section 1 - Architecture Commune

Cette section decrit les composants techniques et architecturaux partages par l'ensemble des sept domaines du Departement 1. L'architecture commune constitue le socle sur lequel chaque domaine specifique viendra s'appuyer, garantissant coherence, securite et performance a l'echelle du systeme complet. Elle couvre le modele de donnees partage, le systeme d'authentification et d'autorisation, la strategie de multi-tenancy, l'architecture evenementielle inter-domaines, et les mecanismes de stockage et de cache.

### 1.1 Vue d'ensemble de l'architecture

L'architecture globale du systeme repose sur un modele en deux couches principales, exploitees via une infrastructure Cloudflare en edge et un backend Supabase en source de verite. Cette organisation permet de tirer parti de la proximite geographique des serveurs Cloudflare (plus de 300 points de presence dans le monde) pour offrir des temps de reponse optimaux, tout en beneficiant de la puissance de PostgreSQL et de l'ecosysteme Supabase pour la gestion des donnees relationnelles complexes propres aux processus RH.

La couche Cloudflare comprend quatre composants essentiels : les Pages (hebergement de l'application Next.js sous forme de PWA), les Workers (API edge pour le routage, la transformation et la validation des requetes), D1 (base SQLite edge pour la mise en cache des donnees frequentes et des dictionnaires), et KV (stockage cle-valeur pour les sessions et les configurations dynamiques). R2 complete cette couche pour le stockage d'objets binaires (documents, avatars, pieces jointes).

La couche Supabase fournit le backend de donnees avec PostgreSQL comme moteur principal, complete par Auth (authentification JWT/OAuth), Storage (stockage de fichiers volumes), Realtime (synchronisation en temps reel via WebSocket) et les Edge Functions (fonctions serverless proches des donnees). L'isolation multi-tenant est assuree par des politiques RLS appliquees uniformement sur toutes les 620 tables du systeme.

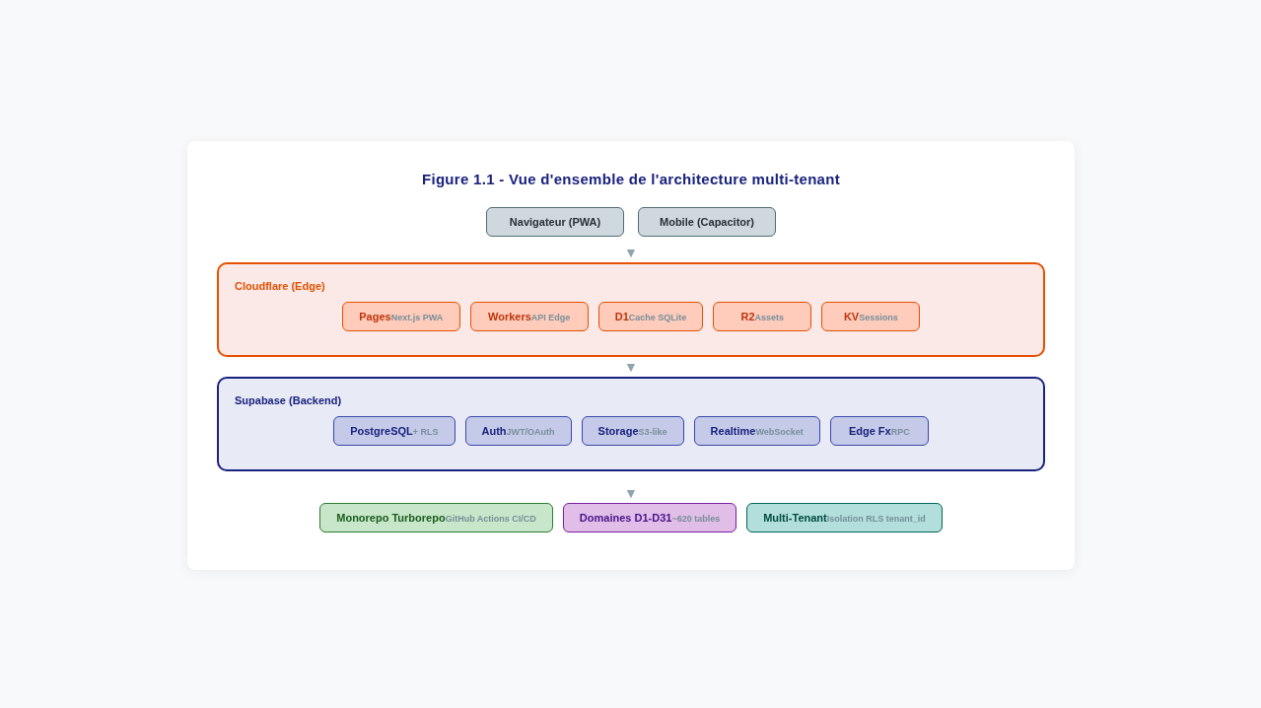


Figure 1.1 - Vue d'ensemble de l'architecture multi-tenant (Supabase + Cloudflare)

1.2 Modele de donnees partage

Le modele de donnees partage constitue le fondement de la coherence informationnelle du Departement 1. Il est compose de cinq tables principales qui sont referencees par l'ensemble des sept domaines : tenants, utilisateurs, employes, structures et postes. Ces tables forment le noyau relationnel du systeme et sont les seules a etre directement accessibles en lecture par tous les domaines. Chaque domaine ajoute ensuite ses propres tables specifiques, liees a ce noyau commun par des cles etrangeres.

| Table        | Role                                                        | Champs cles                                                                               | Utilisee par                   |
|--------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------|
| tenants      | Organisations clientes hebergees sur la plateforme          | id, nom, slug, plan, config (JSONB), actif                                                | Tous les domaines              |
| utilisateurs | Comptes d'accès au système (authentification)               | id, tenant_id, email, mot_de_passe_hash, role_principal, actif                            | Auth + tous domaines           |
| employes     | Donnees d'identite des collaborateurs                       | id, utilisateur_id, tenant_id, matricule, nom, prenom, date_naissance, genre, nationalite | D2, D4, D5, D12, D21, D23, D24 |
| structures   | Unites organisationnelles (departements, services, equipes) | id, tenant_id, parent_id (self-ref), type, code, nom, responsable_id                      | D2, D4, D21, D23, D24          |



| Table  | Role                                       | Champs cles                                                               | Utilisee par               |
|--------|--------------------------------------------|---------------------------------------------------------------------------|----------------------------|
| postes | Postes de travail et repertoires d'emplois | id, tenant_id, structure_id, titre, categorie, echelon, classification_id | D2, D5, D12, D21, D23, D24 |

Tableau 1.1 - Tables partagees du modele de donnees commun



Figure 1.2 - Schema entite-association du modele de donnees partage

1.2.1 Conventions de nommage

Le schema de base de donnees suit des conventions strictes de nommage qui garantissent la lisibilite, la coherence et la maintenance du code a long terme. Ces conventions s'appliquent uniformement a l'ensemble des 620 tables du systeme, quel que soit le domaine ou le departement d'appartenance. Le respect de ces conventions est automatiquement verifie par des regles linter integrees au pipeline CI/CD.

| Element       | Convention                                                       | Exemple                                       |
|---------------|------------------------------------------------------------------|-----------------------------------------------|
| Nom de table  | Serpent_case, pluriel, prefixe par le code domaine si specifique | employes, d02_documents, d05_elements_salaire |
| Cle primaire  | id (UUID ou BIGSERIAL selon le volume attendu)                   | id UUID DEFAULT gen_random_uuid()             |
| Cle etrangere | nom_table_id (singulier)                                         | tenant_id, employe_id, structure_id           |

| Element           | Convention                                       | Exemple                                    |
|-------------------|--------------------------------------------------|--------------------------------------------|
| Timestamps        | created_at, updated_at (TIMESTAMPTZ, auto-geres) | created_at TIMESTAMPTZ DEFAULT now()       |
| Soft delete       | deleted_at (TIMESTAMPTZ NULLABLE)                | WHERE deleted_at IS NULL                   |
| Booleen           | is_ ou has_ prefixe                              | is_actif, has_permis_conduire              |
| Enum / Status     | statut VARCHAR(50) avec CHECK constraint         | statut VARCHAR(50) CHECK (statut IN (...)) |
| JSONB             | Suffixe _json ou _config                         | regles_avancement_json, config_affichage   |
| Index             | idx_{table}_{colonnes}                           | idx_employes_tenant_matricule              |
| Contrainte unique | uq_{table}_{colonnes}                            | uq_employes_tenant_matricule               |

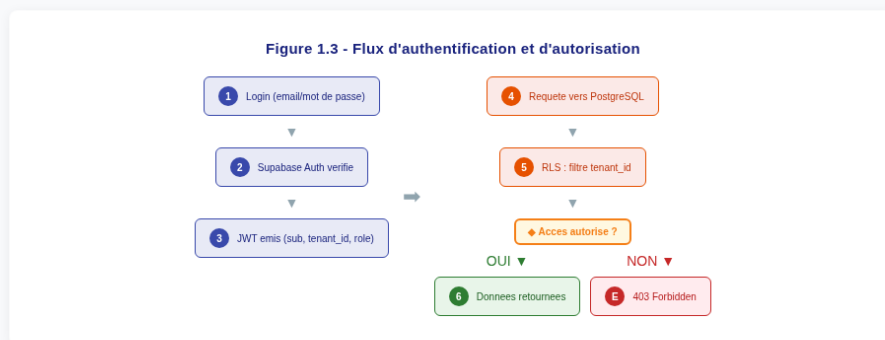
Tableau 1.2 - Conventions de nommage du schema de donnees

## 1.3 Authentification et Autorisation

Le systeme d'authentification et d'autorisation constitue la premiere ligne de defense du SIRH. Il repose sur une approche en deux couches complementaires : l'authentification, qui verifie l'identite de l'utilisateur, et l'autorisation, qui controle ses droits d'accès aux donnees. Cette separation permet de modulariser les mecanismes de securite et de les adapter independamment aux exigences de chaque type d'utilisateur et de chaque tenant.

L'authentification est entierement deleguee a Supabase Auth, qui prend en charge la gestion des identites, la verification des mots de passe (avec hachage bcrypt), la gestion des sessions, et la emission de jetons JWT. Supabase Auth supporte nativement plusieurs methodes d'authentification : email/mot de passe, OAuth (Google, Microsoft, Apple), magic link et SMS. Cette souplesse permet a chaque tenant de choisir les methodes d'authentification les plus adaptees a sa politique de securite.

L'autorisation operant a deux niveaux : un niveau applicatif via les Cloudflare Workers, qui verifient les permissions fonctionnelles de l'utilisateur (role, perimetre d'accès), et un niveau base de donnees via les politiques RLS de PostgreSQL, qui filtrent les donnees accessibles en fonction du tenant\_id. Cette double couche garantit que meme si une requete contourne le niveau applicatif, le niveau base de donnees empechera tout acces non autorise aux donnees d'un autre tenant.



*Figure 1.3 - Flux d'authentification et d'autorisation*

### 1.3.1 Roles et permissions

Le systeme de permissions est base sur un modele RBAC (Role-Based Access Control) enrichi avec des attributs contextuels (ABAC - Attribute-Based Access Control). Chaque utilisateur possede un role principal defini dans la table utilisateurs, et peut se voir attribuer des permissions supplementaires via une table de liaison roles\_permissions. Ce modele hybride permet de definir des regles de acces a la fois generiques (par role) et specifiques (par contexte, par domaine, par perimetre organisationnel).

| Role               | Description                                                    | Acces Lecture | Acces Ecriture | Portee                  |
|--------------------|----------------------------------------------------------------|---------------|----------------|-------------------------|
| <b>super_admin</b> | Administrateur de la plateforme, acces technique complet       | Global        | Global         | Tous les tenants        |
| <b>admin_rh</b>    | Administrateur RH d'un tenant, gestion complete des donnees RH | Tenant        | Tenant         | Son tenant uniquement   |
| <b>responsable</b> | Responsable hierarchique, acces limite a son perimetre         | Perimetre     | Perimetre      | Son service/departement |
| <b>agent_rh</b>    | Agent RH operationnel, acces fonctionnel aux domaines DRH      | Tenant        | Domaines RH    | Son tenant uniquement   |

| Role                 | Description                                   | Acces Lecture  | Acces Ecriture | Portee              |
|----------------------|-----------------------------------------------|----------------|----------------|---------------------|
| <b>collaborateur</b> | Salarie, acces restreint a son propre dossier | Propre dossier | Demandes       | Son dossier employe |

Tableau 1.3 - Roles et niveaux d'accès du systeme

## 1.4 Multi-tenancy par Row Level Security

Le choix de l'isolation multi-tenant par Row Level Security (RLS) constitue l'une des decisions architecturales les plus importantes du systeme. Plutot que de creer une base de donnees ou un schema PostgreSQL par tenant (approche schema-per-tenant), l'architecture utilise un seul projet Supabase shared dans lequel chaque table possede un champ tenant\_id. Les politiques RLS de PostgreSQL filtrent automatiquement chaque requete pour ne retourner que les lignes appartenant au tenant de l'utilisateur connecte.

Cette approche offre plusieurs avantages decisifs par rapport aux alternatives. Premierement, elle simplifie considerablement les operations de deploiement et de maintenance : un seul cluster PostgreSQL a gerer, une seule configuration de Supabase, une seule chaine de connexion. Deuxiemement, elle facilite les mises a jour et les migrations de schema, qui sont appliquees une seule fois pour l'ensemble des tenants. Troisiemement, elle permet une agregation trans-tenant des donnees pour le compte super\_admin, utile pour le monitoring, la facturation et l'analytique globale de la plateforme.

La mise en oeuvre technique repose sur une fonction PostgreSQL qui extrait le tenant\_id du JWT de l'utilisateur connecte. Cette fonction est appelee par chaque politique RLS de chaque table, garantissant un filtrage transparent et automatique sans intervention du code applicatif. Le tableau ci-dessous detaille le mecanisme de creation des politiques RLS pour les differentes operations CRUD.

| Operation     | Politique RLS                                                                  | Description                                                    | Exemple SQL simplifie                                                                            |
|---------------|--------------------------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>SELECT</b> | Politique de lecture : l'utilisateur ne peut lire que les lignes de son tenant | Le tenant_id du JWT doit correspondre au tenant_id de la ligne | CREATE POLICY sel_tenant ON employes FOR SELECT USING (tenant_id = get_current_tenant_id())      |
| <b>INSERT</b> | Politique d'insertion : le tenant_id est automatiquement injecte               | Le tenant_id de la nouvelle ligne est force au tenant du JWT   | CREATE POLICY ins_tenant ON employes FOR INSERT WITH CHECK (tenant_id = get_current_tenant_id()) |
| <b>UPDATE</b> | Politique de modification : seules les lignes du tenant sont modifiables       | L'utilisateur ne peut modifier que les lignes de son tenant    | CREATE POLICY upd_tenant ON employes FOR UPDATE USING (tenant_id = get_current_tenant_id())      |

| Operat<br>ion      | Politique RLS                                                                  | Description                                         | Exemple SQL simplifie                                                                              |
|--------------------|--------------------------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>DELE<br/>TE</b> | Politique de suppression :<br>seules les lignes du tenant<br>sont supprimables | Protection contre les<br>suppressions inter-tenants | CREATE POLICY del_tenant ON<br>employees FOR DELETE USING<br>(tenant_id = get_current_tenant_id()) |

Tableau 1.4 - Politiques RLS par operation CRUD

### 1.4.1 Activation et gestion des politiques RLS

L'activation des RLS est une operation obligatoire sur chaque table du systeme. Sans cette activation, les politiques RLS sont ignorees par PostgreSQL et les donnees seraient accessibles sans filtrage. Le pipeline CI/CD integre un controle automatique qui verifie que chaque table nouvelle ou modifiee possede bien l'attribut `ENABLE ROW LEVEL SECURITY` active et qu'au moins une politique RLS est definie pour chaque operation (`SELECT`, `INSERT`, `UPDATE`, `DELETE`). Ce controle est implemente sous forme de test dans GitHub Actions et bloque tout deploiement ne respectant pas cette regle.

| Etape                  | Commande / Action                                               | Responsable                 | Verification                  |
|------------------------|-----------------------------------------------------------------|-----------------------------|-------------------------------|
| 1. Creation table      | CREATE TABLE ... avec colonne<br>tenant_id NOT NULL             | Migration SQL du<br>domaine | Test unitaire + CI lint       |
| 2. Activation RLS      | ALTER TABLE xxx ENABLE ROW<br>LEVEL SECURITY                    | Migration SQL du<br>domaine | CI check (psql \d+ xxx)       |
| 3. Creation politiques | CREATE POLICY ... FOR<br>SELECT/INSERT/UPDATE/DELETE            | Migration SQL du<br>domaine | CI check (nb politiques >= 4) |
| 4. Test fonctionnel    | Requetes avec JWT de tenant A et B<br>pour verifier l'isolation | Test d'integration          | Pipeline CI/CD e2e            |
| 5. Deploiement         | supabase db push apres validation CI                            | GitHub Actions              | Lock deploiement si echec     |

Tableau 1.5 - Processus d'activation et de gestion des politiques RLS

## 1.5 Architecture evenementielle inter-domaines

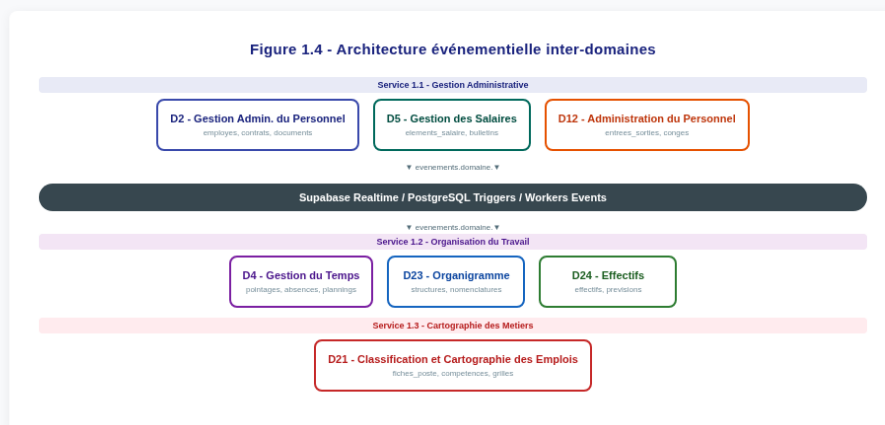
L'un des defis majeurs d'un systeme modulaire compose de 31 domaines interconnectes est de garantir la coherence des donnees sans creer de couplage fort entre les domaines. L'architecture evenementielle repond a ce defi en permettant aux domaines de communiquer de maniere asynchrone et decouplee : un domaine emit un evenement lorsqu'une action importante se produit, et les domaines concernes reagissent a cet evenement en mettant a jour leurs propres donnees.

Le systeme evenementiel du Departement 1 repose sur trois mecanismes complementaires. Les PostgreSQL Triggers capturent les modifications de donnees en temps reel (insertion, mise a jour, suppression) et emettent des evenements dans un canal Supabase Realtime. Les Cloudflare Workers Events ecoutent ces canaux et executent des traitements asynchrones (notification, synchronisation, calcul). Enfin, les fonctions RPC (Remote Procedure Call) de Supabase permettent aux Workers de declencher des traitements cotes base de donnees lorsque necessaire.

Cette architecture offre une resiliance importante : si un Worker tombe en panne, les evenements sont persistes dans la file d'attente Supabase et seront traites a la reprise. De plus, l'absence de couplage direct entre domaines permet de modifier, remplacer ou mettre a jour un domaine sans impacter les autres, tant que le contrat evenementiel (structure du message) reste respecte. Le tableau suivant presente les principaux evenements du Departement 1 et leurs abonnees.

| Evenement source            | Domaine emetteur | Donnees transmises                                       | Abonnees / Actions                                                               |
|-----------------------------|------------------|----------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>employe.cree</b>         | D2               | employe_id, tenant_id, matricule, structure_id, poste_id | D5 (init salaire), D4 (init compteur), D12 (init conges), D21 (lien fiche poste) |
| <b>employe.modifie</b>      | D2               | employe_id, champs_modifies, ancien/nouveau              | D23 (si changement structure), D5 (si changement poste)                          |
| <b>contrat.cree_modifie</b> | D2               | contrat_id, employe_id, type, debut, fin, salaire        | D5 (recalcul paie), D12 (mise a jour droits conges)                              |
| <b>conge.demande_valide</b> | D12              | conge_id, employe_id, dates, statut, validateur_id       | D4 (mise a jour compteur temps), D24 (statistiques)                              |
| <b>pointage.enregistre</b>  | D4               | pointage_id, employe_id, date, heures                    | D12 (calcul solde conges), D5 (heures supplementaires)                           |
| <b>structure.modifiee</b>   | D23              | structure_id, ancien/nouveau parent, responsable         | D2 (affectations), D24 (effectifs par structure)                                 |
| <b>poste.classifie</b>      | D21              | poste_id, classification_id, echelon, grille             | D5 (impact salaire), D2 (historique carriere)                                    |

*Tableau 1.6 - Principaux evenements inter-domaines du Departement 1*



*Figure 1.4 - Architecture evenementielle inter-domaines du Departement 1*

## 1.6 Stockage et Cache

Le systeme de stockage et de cache est conçu pour optimiser les performances de l'application tout en minimisant les coûts d'infrastructure. Il repose sur une stratégie de stockage multiniveau où chaque couche est optimisée pour un type d'utilisation spécifique : les données fréquentes et à faible latence en edge (Cloudflare KV/D1), les assets statiques sur le CDN, les documents et fichiers volumes dans R2 et Supabase Storage, et les données relationnelles complètes dans PostgreSQL.

La couche edge (Cloudflare KV et D1) joue un rôle crucial dans l'optimisation des performances. Les sessions utilisateur, les tokens d'authentification, les configurations de tenant et les dictionnaires de référence (pays, nationalités, types de contrat, etc.) sont mis en cache dans KV avec un temps de réponse inférieur à 5 millisecondes. Les données plus complexes mais fréquemment consultées (comme les listes d'employés, les plannings ou les résumés de paie) sont mises en cache dans D1 (SQLite edge) avec un temps de réponse inférieur à 10 millisecondes. Ce cache est automatiquement invalide lorsqu'une modification est détectée dans la base PostgreSQL source.

Le stockage des fichiers et documents est réparti entre R2 (Cloudflare) pour les assets de l'application (images, logos, templates) et les petits documents, et Supabase Storage pour les documents RH volumes (contrats, bulletins de paie, pièces justificatives) qui nécessitent un contrôle d'accès granulaire basé sur les RLS. Cette répartition permet de bénéficier du CDN Cloudflare pour les assets publics tout en conservant un contrôle de sécurité strict sur les documents sensibles.

| Solution         | Type            | Cas d'usage                                             | Latence | Securite                     |
|------------------|-----------------|---------------------------------------------------------|---------|------------------------------|
| Cloudflare KV    | Cle-valeur      | Sessions, tokens, config tenant, dictionnaires          | <5ms    | Chiffre, isole par tenant    |
| Cloudflare D1    | SQLite edge     | Cache requetes, listes referencees, pre-calculs         | <10ms   | Repliquee, pas de PII direct |
| Cloudflare R2    | S3-compatible   | Assets statiques, logos, templates, avatars             | ~30ms   | Public ou signe URL          |
| Supabase Storage | S3 (PostgreSQL) | Documents RH, contrats, bulletins, pieces jointes       | ~100ms  | RLS, bucket par tenant       |
| PostgreSQL       | Relationnel     | Donnees transactionnelles, 620 tables, source de verite | ~50ms   | RLS, chiffrement at-rest     |
| CDN Cache        | Statique        | JS/CSS bundles, images, polices, PWA shell              | <1ms    | Immutable, fingerprinte      |

Tableau 1.7 - Solutions de stockage et de cache



Figure 1.5 - Architecture de stockage et de cache multiniveau

1.6.1 Strategie d'invalidation du cache

L'invalidation du cache est un aspect critique du systeme de stockage multiniveau. Une strategie d'invalidation mal concue peut entrainer des incoherences de donnees visibles par les utilisateurs, tandis qu'une invalidation



trop agressive peut negatiger les benefices de performance du cache. Le systeme utilise une approche hybride combinee qui s'appuie sur trois mecanismes complementaires : l'invalidation basee sur les evenements (event-driven), l'invalidation par expiration temporelle (TTL), et l'invalidation proactive (pre-warming).

| Mecanisme           | Trigger                                                | Portee                                                     | Exemple                                                                    |
|---------------------|--------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Event-driven</b> | Evenement PostgreSQL trigger (INSERT/UPDATE/DELETE)    | La cle de cache correspondant a la table/ligne modifiee    | Modification d'un salaire (D5) invalide le cache D5bulletins_employe_{id}  |
| <b>TTL</b>          | Expiration automatique apres une duree configuree      | Les cle expirees sont automatiquement supprimees           | Dictionnaires de reference (pays, nationalites) : TTL = 24h                |
| <b>Pre-warming</b>  | Chargement anticipe apres invalidation ou au demarrage | Reconstruction asynchrone du cache sans bloquer la requete | Liste des employes d'un service rechargee en background apres un transfert |
| <b>Manual purge</b> | Action manuelle de l'administrateur (API admin)        | Purge complete ou selective d'un tenant ou d'un domaine    | Reinitialisation complete du cache apres une migration importante          |

*Tableau 1.8 - Strategie d'invalidation du cache multiniveau*